

Python pour le trading — Document 4/8

La concurrence et l'asynchrone

1. Deux mots à distinguer : concurrence et parallélisme

Avant tout, deux termes qu'on confond souvent et qu'un entretien aime séparer. La **concurrence** (concurrency) consiste à **gérer** plusieurs tâches qui progressent sur une même période, en alternant rapidement entre elles — même sur un seul cœur. Le **parallélisme** (parallelism) consiste à **exécuter** réellement plusieurs tâches au même instant, ce qui exige plusieurs cœurs physiques.

Image : un cuisinier seul qui surveille trois plats en passant de l'un à l'autre fait de la **concurrence** (une seule paire de mains, mais rien ne stagne). Trois cuisiniers travaillant chacun sur un plat font du **parallélisme** (trois paires de mains en même temps).

En Python, `threading` et `asyncio` font de la **concurrence** (idéale pour l'attente), tandis que `multiprocessing` apporte du vrai **parallélisme** (idéal pour le calcul). La section suivante explique pourquoi cette frontière existe.

2. Le problème : I/O-bound contre CPU-bound

Avant de choisir un outil de concurrence, il faut comprendre la nature de la tâche. Tout le reste du document en dépend. On distingue deux grands cas selon ce qui limite la tâche : l'attente, ou le calcul.

2.1 Une tâche I/O-bound : elle attend

« I/O » signifie *input/output* (entrées-sorties) : tout ce qui sort du processeur, comme le réseau, le disque, une API. Une tâche I/O-bound passe l'essentiel de son temps à **attendre** une réponse extérieure. Pendant cette attente, le processeur ne fait rien — il est oisif.

```
import time

def recuperer_prix(symbole):
    # Exemple I/O-bound : on demande un prix à une API
    print(f"{symbole} : requête envoyée, on attend la réponse...")
    time.sleep(2)          # 2 secondes d'ATTENTE réseau (le CPU ne fait rien)
    print(f"{symbole} : réponse reçue")
    return 5234.50

# Récupérer 3 prix l'un après l'autre prend 3 x 2 = 6 secondes,
# alors que le processeur passe ces 6 secondes à ne rien faire !
recuperer_prix("ES")
recuperer_prix("NQ")
recuperer_prix("CL")
```

Ici, `time.sleep(2)` simule une attente réseau de 2 secondes. Les trois appels à la suite prennent 6 secondes — mais pendant tout ce temps, le processeur est inactif, il attend juste. C'est du gâchis : on pourrait lancer les trois requêtes **en même temps** et attendre les trois réponses ensemble, en 2 secondes au lieu de 6.

Exemples concrets en trading : attendre la réponse d'un courtier, recevoir un flux de prix par le réseau, lire un gros fichier d'historique sur le disque, interroger une base de données.

2.2 Une tâche CPU-bound : elle calcule

À l'inverse, une tâche CPU-bound passe son temps à **calculer**. Le processeur est à fond, il n'y a aucune attente, aucun temps mort à récupérer.

```
def calculer_indicateurs(prix):  
    # Exemple CPU-bound : un calcul intensif, le processeur travaille à 100 %  
    total = 0  
    for i in range(100_000_000):      # 100 millions d'opérations  
        total += i * 1.01  
    return total  
# Pendant tout ce temps, le processeur CALCULE sans interruption.
```

Cette boucle fait travailler le processeur sans relâche. Il n'y a **pas** de temps mort à exploiter. Pour aller plus vite, la seule solution est d'avoir **plusieurs cœurs** qui calculent en parallèle, chacun sur une partie du travail.

Exemples concrets en trading : entraîner un modèle de machine learning, calculer des indicateurs complexes sur des millions de barres, lancer un backtest lourd.

2.3 Pourquoi cette distinction décide de tout

Tâche I/O-bound (attente) -> on récupère le temps d'attente :
THREADING ou ASYNCIO

Tâche CPU-bound (calcul) -> il faut de vrais cœurs en plus :
MULTIPROCESSING

C'est la décision d'architecture la plus importante. Mais pour comprendre **pourquoi** le threading marche pour l'attente et pas pour le calcul, il faut connaître une particularité de Python : le GIL.

Pourquoi ça compte. *Se tromper de catégorie est l'erreur n°1 en concurrence. Savoir d'abord si une tâche attend ou calcule oriente vers la bonne solution — la première question qu'on se pose en concevant un système temps réel.*

3. Le GIL : la particularité de Python

Le **GIL** (*Global Interpreter Lock*, « verrou global de l'interpréteur ») est un mécanisme **interne** de Python qui n'autorise qu'**un seul thread à exécuter du code Python à la fois**. Même avec 16 cœurs, le code Python de plusieurs threads ne tourne pas vraiment en parallèle : les threads se relaient pour tenir ce verrou unique.

3.1 Le GIL est conceptuel : on ne le code pas

Point important : le GIL n'est pas une fonction qu'on appelle ni une ligne qu'on écrit. C'est un mécanisme **invisible** intégré à l'interpréteur Python, qui agit en coulisse. On ne peut donc pas « coder avec le GIL » : on peut seulement observer ses **effets**. Les exemples ci-dessous ne montrent pas le GIL lui-même (il est implicite), mais du code dont le **comportement révèle** sa présence.

3.2 Une image pour comprendre

Imagine un seul micro pour plusieurs personnes : une seule peut parler à la fois, elle passe le micro à la suivante. Le GIL est ce micro. Plusieurs threads existent, mais un seul « parle » (exécute du code Python) à un instant donné.

3.3 L'effet sur un calcul (lien avec le threading)

Voici du code CPU-bound dont le résultat **révèle** le GIL : un calcul lourd, lancé avec 4 threads, ne va **pas** plus vite qu'avec un seul.

```
import threading, time

def calcul_lourd():
    total = 0
    for i in range(50_000_000):
        total += i
    return total

debut = time.time()
threads = [threading.Thread(target=calcul_lourd) for _ in range(4)]
for th in threads: th.start()
for th in threads: th.join()
print(f"4 threads : {time.time() - debut:.1f}s")
# Résultat : à peu près le même temps qu'un seul calcul, parfois PIRE.
# Le GIL empêche les 4 threads de calculer vraiment en même temps.
```

Les 4 threads se disputent le « micro » (le GIL) : un seul calcule à la fois, donc aucun gain. C'est le **lien GIL ↔ threading** : pour du calcul **en Python pur**, le threading est inutile. C'est exactement pourquoi on a besoin du multiprocessing (section 5).

3.4 Nuance importante : le code natif libère le GIL

Il y a une exception capitale, souvent ignorée. Le GIL ne verrouille que l'exécution de **code Python** (le *bytecode* interprété). Or certaines opérations « de calcul » ne s'exécutent **pas** en Python : elles sont déléguées à du code natif compilé (C ou Fortran). Pendant cette délégation, ces bibliothèques **relâchent le GIL**, et le calcul peut alors utiliser plusieurs cœurs — même avec des threads.

Comparons deux calculs en apparence semblables, mais radicalement différents face au GIL.

```
# CAS 1 : boucle Python pure -> GARDE le GIL -> le threading n'aide PAS
def calcul():
    total = 0
    for i in range(10_000_000):    # chaque tour est du bytecode Python
        total += i

# CAS 2 : opération NumPy -> code natif (BLAS) -> RELÂCHE le GIL
import numpy as np
A = np.random.rand(5000, 5000)
```

```
B = np.random.rand(5000, 5000)
C = A @ B                                # multiplication matricielle en code natif
```

Dans le **cas 1**, chaque itération est du bytecode Python : le GIL est tenu en permanence, et plusieurs threads ne donneraient aucun gain. Dans le **cas 2**, l'opération `A @ B` (multiplication matricielle) est confiée à une bibliothèque native optimisée — **BLAS**, **MKL** ou **OpenBLAS** — écrite en C/Fortran. Cette bibliothèque relâche le GIL pendant le calcul et parallélise elle-même sur plusieurs cœurs. Autrement dit, une partie de ce qui ressemble à du CPU-bound se comporte, du point de vue du GIL, comme de l'I/O : le thread « rend la main » pendant que le code natif travaille.

Conséquence pratique. La règle « threading inutile pour le CPU » vaut pour le calcul en **Python pur**. Dès qu'on s'appuie sur NumPy, Pandas, PyTorch ou TensorFlow (qui reposent tous sur du code natif), une bonne part du calcul échappe au GIL et exploite déjà plusieurs cœurs, sans qu'on ait à gérer le multiprocessing. C'est l'une des raisons pour lesquelles la vectorisation NumPy (document 5) est si efficace : non seulement elle évite l'interpréteur Python, mais elle peut aussi paralléliser au niveau natif.

***Pourquoi ça compte.** Beaucoup de calculs de trading passent par NumPy ou des frameworks ML, qui libèrent le GIL et parallélisent en code natif. Comprendre cette nuance évite de croire à tort que tout calcul Python est bridé par le GIL — et explique pourquoi on n'a pas toujours besoin du multiprocessing si on s'appuie sur ces bibliothèques.*

3.5 L'effet sur l'attente (lien avec threading et asyncio)

La nuance essentielle : quand un thread **attend** (réseau, disque), il n'exécute pas de code Python, donc il **relâche le micro**, laissant un autre avancer. C'est pourquoi le threading **fonctionne** pour l'I/O-bound — et c'est aussi ce qui rend asyncio possible.

```
Thread qui CALCULE  -> garde le GIL      -> bloque les autres -> AUCUN parallélisme
Thread qui ATTEND   -> relâche le GIL     -> les autres avancent -> concurrence UTILE
```

Résumé du lien GIL ↔ outils : pour l'**attente** (I/O), le GIL est relâché, donc threading et asyncio fonctionnent. Pour le **calcul** (CPU), le GIL bloque, donc il faut le multiprocessing — où chaque **processus** a son **propre** GIL (quatre processus = quatre micros, qui parlent vraiment en même temps).

3.6 Et le « Python sans GIL » (free-threading) ?

Une évolution récente, qui revient désormais en entretien. Depuis Python 3.13 (2024) existe une build **expérimentale sans GIL** dite *free-threaded* (issue du PEP 703), passée en statut « officiellement supporté mais optionnel » avec Python 3.14. Dans cette variante, le verrou unique est désactivé, et plusieurs threads peuvent enfin exécuter du bytecode Python **vraiment en parallèle**, y compris pour du CPU-bound.

Trois précisions à retenir pour ne pas dire de bêtise. D'abord, ce n'est **pas la build par défaut** : il faut installer un interpréteur séparé (souvent noté `python3.13t`), et tout l'écosystème de bibliothèques C n'est pas encore prêt. Ensuite, le code mono-thread y est légèrement **plus lent** (de l'ordre de 5 à 10 %), à cause des verrous fins ajoutés. Enfin, dès que le GIL disparaît, on retombe sur les **conditions de course** (section 6) : il faut alors protéger soi-même les données partagées. En 2026, le GIL reste donc bien présent dans la build standard ; tout ce document s'applique tel quel. Le free-threading est l'avenir probable, pas le présent par défaut.

Pourquoi ça compte. Le GIL est LA question piège des entretiens Python. Le comprendre — savoir qu'il est conceptuel et non une ligne de code, et connaître l'arrivée du mode sans GIL — évite de perdre des heures à mettre des threads sur un calcul, et explique pourquoi *threading* sert l'I/O tandis que *multiprocessing* sert le CPU.

4. Le threading : pour les tâches qui attendent

Un **thread** est un fil d'exécution au sein du programme. Comme on vient de le voir avec le GIL, le *threading* est utile pour les tâches **I/O-bound** : pendant qu'un thread attend, les autres avancent. Reprenons l'exemple des trois prix (6 secondes en série) et accélérons-le.

```
import threading
import time

def recuperer_prix(symbole):
    print(f"{symbole} : on attend...")
    time.sleep(2)                # attente réseau simulée
    print(f"{symbole} : reçu")

# Un thread par symbole : les trois attentes se CHEVAUCHENT
threads = []
for symbole in ["ES", "NQ", "CL"]:
    th = threading.Thread(target=recuperer_prix, args=(symbole,))
    th.start()                  # démarre le thread (ne bloque pas)
    threads.append(th)

for th in threads:
    th.join()                   # attend la fin de chaque thread

# Durée totale : ~2 secondes (et non 6), car les attentes sont simultanées
```

`Thread(target=..., args=...)` crée un thread qui exécutera la fonction. `start()` le lance **sans attendre** qu'il finisse, donc les trois démarrent presque ensemble. Les trois attentes (pendant lesquelles le GIL est relâché) se déroulent **en parallèle**, d'où 2 secondes au lieu de 6. `join()` attend ensuite que chaque thread soit terminé. Le code de la fonction n'a **rien de spécial** : c'est la souplesse du *threading*, il accepte du code ordinaire.

Pourquoi ça compte. Suivre plusieurs flux de marché ou interroger plusieurs courtiers en même temps est exactement ce cas : de l'attente réseau qu'on fait se chevaucher. Le *threading* rend le système réactif sans réécrire la logique existante.

5. Données partagées, conditions de course et verrous

Le *threading* apporte un danger que l'exemple précédent évitait : dès que plusieurs threads **modifient une même variable**, le résultat peut devenir faux et imprévisible. C'est la **condition de course** (*race condition*), l'un des bugs les plus redoutés — et un grand classique d'entretien.

5.1 Le problème : une condition de course

Imaginons un compteur partagé que 4 threads incrémentent chacun 100 000 fois. On s'attend à 400 000. On obtient souvent **moins**, et un nombre différent à chaque exécution.

```

import threading

solde = 0                                # variable PARTAGÉE entre les threads

def depoter():
    global solde
    for _ in range(100_000):
        solde += 1                        # lecture + addition + écriture : NON atomique

threads = [threading.Thread(target=deposer) for _ in range(4)]
for th in threads: th.start()
for th in threads: th.join()

print(solde)
# Attendu : 400000
# Obtenu : par ex. 312774, puis 287310... un résultat FAUX et VARIABLE

```

Pourquoi ? Parce que `solde += 1` n'est pas une opération **atomique** : Python la décompose en trois étapes — lire `solde`, lui ajouter 1, réécrire le résultat. Le GIL peut basculer d'un thread à l'autre **entre** ces étapes. Deux threads lisent alors la même valeur (disons 42), ajoutent chacun 1, et réécrivent 43 : deux incréments n'en produisent qu'un. Des incréments sont « perdus », d'où le total trop faible.

Note importante : le GIL n'empêche **pas** les conditions de course. Il garantit qu'une seule *instruction bytecode* s'exécute à la fois, mais une ligne comme `solde += 1` représente **plusieurs** instructions ; le basculement peut survenir au milieu. C'est une confusion fréquente à éviter.

5.2 La solution : un verrou (Lock)

Un **verrou** (*Lock*) garantit qu'un seul thread à la fois exécute la portion « sensible » du code (la **section critique**). Les autres attendent leur tour. On l'utilise avec `with`, qui prend le verrou à l'entrée du bloc et le relâche à la sortie (même en cas d'erreur — exactement le context manager du document 3).

```

import threading

solde = 0
verrou = threading.Lock()                # le verrou qui protège "solde"

def depoter():
    global solde
    for _ in range(100_000):
        with verrou:                      # un seul thread entre ici à la fois
            solde += 1                    # section critique : maintenant sûre

threads = [threading.Thread(target=deposer) for _ in range(4)]
for th in threads: th.start()
for th in threads: th.join()

print(solde)

```

Obtenu : 400000, à chaque exécution. Correct et reproductible.

Avec `with verrou:`, le trio lire-ajouter-écrire devient indivisible du point de vue des autres threads : plus aucun incrément n'est perdu. Le prix à payer est une légère **sérialisation** (les threads font la queue sur la section critique), donc on garde celle-ci aussi **courte** que possible.

5.3 Le piège du verrou : l'interblocage (deadlock)

Les verrous introduisent leur propre danger. Un **interblocage** (*deadlock*) survient quand deux threads attendent chacun un verrou que l'autre détient : aucun ne peut avancer, le programme se fige pour toujours.

```
verrou_a = threading.Lock()
verrou_b = threading.Lock()

def tache_1():
    with verrou_a:
        with verrou_b:            # thread 1 veut B (déjà pris par thread 2)
            ...

def tache_2():
    with verrou_b:
        with verrou_a:            # thread 2 veut A (déjà pris par thread 1)
            ...

# Si les deux démarrent ensemble : chacun tient un verrou et attend l'autre.
# -> blocage total et définitif.
```

La parade la plus simple : **toujours acquérir les verrous dans le même ordre** (ici, A puis B partout). On limite aussi le nombre de verrous, et on préfère souvent une **queue** (section 8) qui gère la synchronisation pour nous, évitant d'avoir à poser des verrous à la main.

***Pourquoi ça compte.** Dans un système de trading multi-thread (un solde, un carnet d'ordres, une position partagée), une condition de course corrompt silencieusement des données financières. Savoir reconnaître le problème, le corriger avec un Lock, et éviter l'interblocage est une compétence attendue — et une question d'entretien quasi systématique.*

6. Le multiprocessing : pour les vrais calculs

Pour une tâche CPU-bound, le threading est inutile (GIL). La solution est le **multiprocessing** : lancer plusieurs **processus** séparés, chacun avec son propre interpréteur et son propre GIL, donc capables de calculer vraiment en parallèle sur plusieurs cœurs.

```
from multiprocessing import Pool

def calculer_indicateurs(donnees):
    return analyse_complexe(donnees)    # CPU-bound, calcul lourd

if __name__ == "__main__":
    historiques = [hist_es, hist_nq, hist_cl, hist_gc]
    with Pool(4) as pool:                # 4 processus
        resultats = pool.map(calculer_indicateurs, historiques)
```

```
# Les 4 calculs tournent en parallèle, chacun sur un cœur
```

Pool(4) crée quatre processus, pool.map répartit les éléments entre eux. La ligne `if __name__ == "__main__"` est **obligatoire** (sans elle, les processus enfants relanceraient le programme en boucle). Et comme les processus **ne partagent pas leur mémoire**, échanger des données entre eux a un coût (il faut les sérialiser et les transférer).

6.1 C'est le nombre de CŒURS qui compte, pas de « CPU »

Précision matérielle importante. Un ordinateur a presque toujours **une seule puce processeur** (un « CPU »), mais cette puce contient **plusieurs cœurs**, et chaque cœur peut calculer indépendamment. Les machines à plusieurs puces (« 2 CPU », « 4 CPU ») n'existent en pratique que dans les serveurs spécialisés. Pour Python, ce qui détermine le parallélisme réel, c'est le **nombre total de cœurs**, peu importe comment ils sont répartis en puces.

```
import os
os.cpu_count()      # -> nombre total de cœurs disponibles (ex: 4, 8, 16)
```

6.2 Comparaison concrète selon le nombre de cœurs

Supposons 8 calculs lourds, chacun prenant 10 secondes seul, soit **80 secondes en série**. Voyons le temps total selon le matériel et le nombre de processus :

Matériel	Pool()	Ce qui se passe	Temps total
-----	-----	-----	-----
4 cœurs	Pool(4)	4 calculs à la fois, puis 4	~20 s
16 cœurs	Pool(8)	les 8 calculs en même temps	~10 s
16 cœurs	Pool(16)	8 calculs ; 8 cœurs restent oisifs	~10 s
1 cœur	Pool(4)	les "processus" se relaient	~80 s (aucun gain)

Lecture de ce tableau. Avec **4 cœurs**, on traite 4 calculs simultanément, puis les 4 suivants : 2 vagues de 10 s = 20 s. Avec **16 cœurs**, les 8 calculs tiennent en une seule vague = 10 s. Mettre Pool(16) sur seulement 8 calculs n'aide pas plus : il n'y a que 8 calculs à faire, donc 8 cœurs travaillent et les 8 autres restent oisifs. Et avec **1 seul cœur**, le multiprocessing n'apporte **aucun gain** : les processus se relaient sur l'unique cœur, comme en série.

Deux règles à retenir. D'abord, le gain est plafonné par le nombre de cœurs : sur 4 cœurs, au mieux 4 fois plus rapide. Ensuite, créer plus de processus que de cœurs n'accélère pas le calcul pur (les processus en trop attendent un cœur libre) ; en général, on choisit `Pool(os.cpu_count())`, soit autant de processus que de cœurs.

Pourquoi ça compte. Le matériel fixe la limite du parallélisme : connaître le nombre de cœurs de la machine de calcul (et y adapter la taille du Pool) permet d'exploiter pleinement le matériel sans gaspiller. Sur un serveur de recherche à 16 cœurs, un backtest bien parallélisé va jusqu'à 16 fois plus vite — des heures transformées en minutes.

7. asyncio : la concurrence dans un seul thread

L'**asynchrone** avec `asyncio` est une autre façon de gérer les tâches I/O-bound. Comme le `threading`, il sert l'attente (le GIL est relâché), mais avec une approche différente : tout se passe dans **un seul thread**, grâce aux mots-clés `async` et `await`.

7.1 async et await

```
import asyncio

async def recuperer_prix(symbole):          # 'async def' = une COROUTINE
    print(f"{symbole} : on attend...")
    await asyncio.sleep(2)                 # attente NON bloquante ('await')
    print(f"{symbole} : reçu")

async def main():
    # lancer les trois EN CONCURRENCE dans un seul thread
    await asyncio.gather(
        recuperer_prix("ES"),
        recuperer_prix("NQ"),
        recuperer_prix("CL"),
    )

asyncio.run(main())
# Durée : ~2 secondes
```

Une fonction `async def` est une **coroutine** : une fonction qui peut se mettre en pause et reprendre. Le mot `await` marque un point d'attente : « attends le résultat ici, mais pendant ce temps, **rends la main** pour que les autres coroutines avancent ». `asyncio.gather` lance plusieurs coroutines ensemble, et `asyncio.run` démarre la boucle qui les orchestre. Résultat : 2 secondes pour les trois, comme avec le `threading`.

Détail utile : appeler `recuperer_prix("ES")` seul ne l'exécute **pas** — cela crée juste un objet coroutine « en attente ». Une coroutine ne tourne que si on l'*await*, qu'on la passe à `asyncio.gather`, ou qu'on la programme avec `asyncio.create_task(...)` (qui la lance en arrière-plan sans bloquer tout de suite). Oublier d'attendre une coroutine est une erreur classique : le code « ne fait rien » et Python émet un avertissement « coroutine was never awaited ».

7.2 Comment ça marche : la boucle d'événements

`asyncio` repose sur une **boucle d'événements** (*event loop*) : un chef d'orchestre, dans l'unique thread, qui fait tourner les coroutines. Quand une coroutine atteint un `await` (donc une attente), elle rend la main à la boucle, qui lance ou reprend une autre coroutine. Quand l'attente est finie, la boucle revient à la première. Tout cela dans **un seul thread**, sans le coût de créer des threads. Le lien avec le GIL : comme il n'y a qu'un thread, la question du GIL ne se pose même pas — d'où la légèreté d'`asyncio` pour un très grand nombre de tâches.

7.3 Le point de vigilance : tout doit être « async »

Comme tout tient dans un seul thread et que la main n'est rendue qu'aux `await`, si **une seule** opération bloque sans `await`, elle **gèle TOUTES** les autres coroutines.

```
async def recuperer_prix(symbole):
    time.sleep(2)          # ERREUR : time.sleep BLOQUE le thread unique.
    # Aucune autre coroutine ne peut avancer pendant 2 s -> 6 s au total.

async def recuperer_prix(symbole):
```

```
await asyncio.sleep(2)    # CORRECT : rend la main pendant l'attente
```

La règle : en asyncio, **tout le code I/O doit être async de bout en bout**. Pas de `time.sleep`, pas de requests classique, pas de pilote de base de données ordinaire — il faut leurs versions asynchrones (`asyncio.sleep`, `aiohttp`, `aiocpg`). C'est la contrainte propre à asyncio, qu'on compare au threading à la section suivante.

Pourquoi ça compte. *asyncio permet de gérer des milliers de connexions simultanées dans un seul thread, avec très peu de surcharge — d'où sa popularité dans les systèmes de trading modernes connectés à de nombreux marchés. À condition d'utiliser un écosystème async complet.*

8. Threading contre asyncio : comparaison

Les deux servent l'I/O-bound et donnent ici le même résultat (3 prix en 2 s). On les a vus séparément ; comparons-les maintenant point par point, car le choix entre les deux est une vraie décision d'architecture.

8.1 La différence de fond

THREADING : PLUSIEURS threads. Le système d'exploitation bascule de l'un à l'autre, n'importe quand, automatiquement.

ASYNCIO : UN SEUL thread. Les coroutines se passent la main elles-mêmes, UNIQUEMENT aux points marqués 'await'.

Avec le threading, c'est le système d'exploitation qui décide quand basculer entre threads — tu n'as rien à coder de spécial (concurrence *préemptive*). Avec asyncio, les tâches **coopèrent** : elles ne cèdent la main qu'aux `await` (concurrence *coopérative*). D'où la conséquence majeure ci-dessous — et, au passage, le fait qu'en threading on doit penser aux conditions de course (section 5), tandis qu'en asyncio le code entre deux `await` ne peut pas être interrompu, ce qui réduit ce risque.

8.2 L'incompatibilité d'asyncio (que le threading n'a pas)

THREADING : accepte N'IMPORTE QUEL code existant, sans le réécrire. `time.sleep`, requests classique... tout fonctionne.

ASYNCIO : exige du code "async" PARTOUT. Une seule opération bloquante gèle toutes les coroutines. Il faut `aiohttp`, `aiocpg`, etc.

C'est la différence pratique la plus importante. Le threading **tolère** le code ordinaire : tu peux paralléliser une bibliothèque existante sans la modifier. asyncio **impose** un écosystème async complet : si un seul maillon bloque, toute la chaîne gèle. C'est plus puissant à grande échelle, mais plus contraignant.

8.3 Lequel choisir ?

Critère	THREADING	ASYNCIO
-----	-----	-----
Nombre de tâches	dizaines	milliers
Code existant	accepté tel quel	doit être réécrit async

Facilité d'ajout	simple	demande de la rigueur
Surcharge par tâche	moyenne (un thread)	très faible (1 seul thread)
Idéal pour	quelques flux	un très grand nombre de flux

En résumé : **threading** pour un nombre modéré de tâches ou du code existant qu'on ne veut pas réécrire ; **asyncio** pour un très grand nombre de connexions simultanées, quand on accepte d'écrire tout le code en async. Les deux servent l'attente ; aucun n'accélère le calcul pur (pour ça, c'est le multiprocessing).

***Pourquoi ça compte.** Savoir choisir entre `threading` et `asyncio` — et expliquer l'incompatibilité `async` — est exactement le genre de nuance qui distingue un candidat qui maîtrise vraiment la concurrence d'un autre qui en a seulement entendu parler.*

9. concurrent.futures : l'interface unifiée et moderne

En pratique, on écrit rarement `threading.Thread` ou `multiprocessing.Process` à la main pour répartir un lot de tâches. Le module `concurrent.futures` offre une **interface unique** et de plus haut niveau, qui bascule entre threads et processus en changeant **une seule classe**.

9.1 ThreadPoolExecutor (I/O) et ProcessPoolExecutor (CPU)

La même structure sert les deux cas. Pour de l'attente (I/O), on prend `ThreadPoolExecutor` ; pour du calcul (CPU), `ProcessPoolExecutor`. Le reste du code ne change pas.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def recuperer_prix(symbole):
    ... # I/O-bound (attente réseau)

def calculer_indicateurs(donnees):
    ... # CPU-bound (calcul lourd)

# --- Cas I/O : des threads ---
with ThreadPoolExecutor(max_workers=8) as executor:
    prix = list(executor.map(recuperer_prix, ["ES", "NQ", "CL"]))

# --- Cas CPU : des processus (il suffit de changer la classe) ---
with ProcessPoolExecutor(max_workers=4) as executor:
    resultats = list(executor.map(calculer_indicateurs, historiques))
```

Le `with` crée le pool de workers et l'arrête proprement à la sortie. `executor.map(fonction, iterable)` applique la fonction à chaque élément en répartissant le travail sur les workers, et renvoie les résultats **dans l'ordre d'entrée**. Pour passer de l'I/O au CPU, on ne touche qu'au nom de la classe — c'est tout l'intérêt.

9.2 submit et les objets Future

Quand on veut lancer des tâches **hétérogènes** (pas le même travail pour chacune) ou récupérer les résultats **dès qu'ils arrivent**, on utilise `submit`, qui renvoie un **Future** : un objet « promesse » représentant un résultat pas encore disponible.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
```

```

with ThreadPoolExecutor(max_workers=8) as executor:
    futures = {executor.submit(recuperer_prix, s): s for s in ["ES", "NQ", "CL"]}
    for future in as_completed(futures):      # au fur et à mesure des fins
        symbole = futures[future]
        prix = future.result()                # récupère la valeur (ou relève
l'erreur)
        print(f"{symbole} -> {prix}")

```

submit planifie un appel et rend immédiatement un Future. future.result() donne la valeur quand elle est prête (et **relève l'exception** si la tâche a échoué, ce qui facilite la gestion d'erreurs). as_completed itère les futures dans l'ordre où ils **se terminent**, pas dans l'ordre de soumission — pratique pour traiter les réponses les plus rapides en premier.

***Pourquoi ça compte.** concurrent.futures est l'API qu'on utilise réellement en production, et celle qu'on attend de te voir manier en entretien : un seul style de code pour le threading et le multiprocessing, une gestion d'erreurs propre via les Future, et la possibilité de traiter les résultats à mesure qu'ils arrivent.*

10. Les files d'attente (queues)

Quand plusieurs tâches concurrentes doivent **s'échanger des données**, la **file d'attente** (queue) est l'outil sûr — souvent plus simple et moins risqué que poser des verrous à la main (section 5). C'est le motif **producteur-consommateur** : un côté produit (les ticks arrivent), l'autre consomme (on les traite), via une file qui sert de tampon. Important : il existe une queue **différente** pour chaque outil de concurrence — threading, asyncio et multiprocessing — car chacun a ses contraintes. On les voit toutes les trois.

10.1 Le principe commun

PRODUCTEUR	FILE D'ATTENTE	CONSOMMATEUR
(reçoit les ticks) ->	[t1, t2, t3, t4, ...] ->	(analyse les ticks)
rapide	tampon qui absorbe	plus lent
	les écarts de rythme	

Le producteur **dépose** (put), le consommateur **retire** (get). Si la file est vide, le consommateur attend ; si le producteur va trop vite, les éléments s'accumulent sans être perdus. La file gère **elle-même** la synchronisation (verrous internes), ce qui évite les conditions de course sans qu'on ait à écrire de Lock. Voyons la version propre à chaque outil.

10.2 Avec le threading : queue.Queue

```

import queue
import threading

file = queue.Queue()                # file sûre ENTRE THREADS
FIN = object()                      # sentinelle : signal "plus rien à traiter"

def producteur():
    for tick in flux_de_marche():
        file.put(tick)              # dépose

```

```

        file.put(FIN)                                # prévient le consommateur que c'est fini

def consommateur():
    while True:
        tick = file.get()                            # retire (attend si vide)
        if tick is FIN:                               # on a reçu le signal de fin -> on sort
            break
        analyser(tick)

threading.Thread(target=producteur).start()
threading.Thread(target=consommateur).start()

```

`queue.Queue` est conçue pour les threads : elle gère les verrous internes pour qu'un put et un get simultanés ne se corrompent pas. Noter la **sentinelle** `FIN` : sans un signal explicite de fin, le consommateur resterait bloqué pour toujours sur une file vide. C'est le motif standard pour arrêter proprement un consommateur `while True`. C'est la queue à utiliser quand on a choisi le `threading` (section 4).

10.3 Avec asyncio : `asyncio.Queue`

En `asyncio`, on ne peut pas utiliser `queue.Queue` : ses `put/get` sont bloquants et gêleraient le thread unique (rappel de la section 7.3). Il faut la version **async**, dont les opérations s'utilisent avec `await`.

```

import asyncio

async def producteur(file):
    for tick in flux_de_marche():
        await file.put(tick)        # 'await' : ne bloque pas les autres coroutines
    await file.put(None)           # sentinelle de fin

async def consommateur(file):
    while True:
        tick = await file.get()    # 'await' : rend la main si la file est vide
        if tick is None:           # signal de fin
            break
        analyser(tick)

async def main():
    file = asyncio.Queue()         # file pour COROUTINES
    await asyncio.gather(
        producteur(file),
        consommateur(file),
    )

asyncio.run(main())

```

La structure est la même, mais `await file.put(...)` et `await file.get()` rendent la main pendant l'attente, respectant la règle « tout `async` » d'`asyncio`. C'est la queue à utiliser quand on

a choisi `asyncio` (section 7). Utiliser la mauvaise (une `queue.Queue` classique ici) gèlerait tout — une erreur fréquente.

10.4 Avec le multiprocessing : multiprocessing.Queue

Entre **processus**, le défi est différent : les processus ne partagent pas leur mémoire (section 6). Une queue ordinaire ne marcherait pas, car chaque processus aurait sa propre copie. La `multiprocessing.Queue` est spéciale : elle **transporte les données d'un processus à l'autre** (en les sérialisant en coulisse).

```
from multiprocessing import Process, Queue

def producteur(file):
    for tick in flux_de_marche():
        file.put(tick)          # envoyé vers l'AUTRE processus
        file.put(None)         # sentinelle de fin

def consommateur(file):
    while True:
        tick = file.get()
        if tick is None:
            break
        analyser(tick)

if __name__ == "__main__":
    file = Queue()             # file ENTRE PROCESSUS
    p1 = Process(target=producteur, args=(file,))
    p2 = Process(target=consommateur, args=(file,))
    p1.start(); p2.start()
    p1.join(); p2.join()       # on attend la fin des deux processus
```

Ici, producteur et consommateur sont des **processus** distincts (pas des threads). La `multiprocessing.Queue` fait voyager chaque tick d'un processus à l'autre. C'est plus coûteux qu'une queue `Queue` (il faut sérialiser et transférer les données), donc on l'utilise quand on a vraiment besoin de processus séparés — typiquement pour combiner réception et calcul lourd. Noter le `join()` final : sans lui, le programme principal pourrait se terminer avant les processus enfants.

10.5 Quelle queue pour quel outil ?

Outil de concurrence	Queue à utiliser	put/get
threading	queue.Queue	bloquants (classiques)
asyncio	asyncio.Queue	avec 'await'
multiprocessing	multiprocessing.Queue	transfère entre processus

La règle : **la queue doit correspondre à l'outil de concurrence**. Mélanger les deux (une queue `Queue` dans du code `asyncio`, par exemple) provoque des blocages ou des erreurs. C'est un point de vigilance classique.

Pourquoi ça compte. Découpler la réception des données de leur traitement est un motif central des systèmes temps réel. La file absorbe les rafales du marché, garantit qu'aucun tick n'est perdu, et permet de répartir la charge. Savoir choisir la bonne queue selon l'outil (threading, asyncio ou multiprocessing) est exactement le genre de détail qui compte dans un moteur recevant un order flow intense — comme ton bot.

Récapitulatif

- 1. Concurrency vs parallélisme :** gérer plusieurs tâches en alternance (un cœur suffit) contre les exécuter réellement en même temps (plusieurs cœurs).
- 2. I/O-bound vs CPU-bound :** la tâche attend (réseau, disque) ou elle calcule. Cette nature dicte l'outil.
- 3. Le GIL :** mécanisme interne (non codable) qui limite l'exécution Python à un thread ; relâché pendant l'attente et par le code natif. D'où : threading/asyncio pour l'I/O, multiprocessing pour le CPU. La build sans GIL (3.13+) existe mais reste optionnelle.
- 4. Threading :** fait se chevaucher les attentes (I/O) ; accepte le code existant tel quel.
- 5. Conditions de course et verrous :** des threads qui modifient une donnée partagée la corrompent ; un Lock (with verrou:) protège la section critique ; attention à l'interblocage.
- 6. Multiprocessing :** plusieurs cœurs, vrai parallélisme de calcul (CPU) ; le gain est plafonné par le nombre de cœurs.
- 7. asyncio :** des milliers de tâches I/O dans un seul thread (boucle d'événements) ; mais tout le code doit être async.
- 8. Threading vs asyncio :** threading tolère le code existant (préemptif) ; asyncio passe à l'échelle mais impose l'async partout (coopératif).
- 9. concurrent.futures :** interface unifiée (ThreadPoolExecutor / ProcessPoolExecutor) ; objets Future, submit, as_completed.
- 10. Queues :** échange sûr de données (producteur-consommateur) ; une version par outil (queue.Queue, asyncio.Queue, multiprocessing.Queue).

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous.

Sur les fondations (concurrency, I/O, CPU, GIL)

Q1. Différence entre concurrence et parallélisme ?

Réponse. La concurrence gère plusieurs tâches qui progressent en alternance (possible sur un seul cœur) ; le parallélisme les exécute réellement au même instant (exige plusieurs cœurs). En Python, threading et asyncio font de la concurrence, multiprocessing du parallélisme.

Q2. Différence entre I/O-bound et CPU-bound, avec un exemple de chaque ?

Réponse. I/O-bound : la tâche attend une ressource extérieure (ex. attendre la réponse d'un courtier). CPU-bound : la tâche calcule sans interruption (ex. entraîner un modèle).

Q3. Qu'est-ce que le GIL et pourquoi ne peut-on pas « le coder » ?

Réponse. Un verrou interne de l'interpréteur qui n'autorise qu'un thread à exécuter du code Python à la fois. Il est conceptuel et invisible : on n'y accède pas par des lignes de code, on observe seulement ses effets (un calcul multi-thread qui ne va pas plus vite).

Q4. Quel est le lien entre le GIL et le choix threading/multiprocessing ?

Réponse. Le GIL bloque le calcul pur multi-thread (un seul thread calcule à la fois), donc threading n'aide pas pour le CPU. Mais il est relâché pendant l'attente, donc threading sert l'I/O. Pour le calcul parallèle, il faut multiprocessing, où chaque processus a son propre GIL.

Q5. Une multiplication matricielle NumPy ($A @ B$) bénéficie-t-elle du threading ? Pourquoi ?

Réponse. Oui, en partie. Le GIL ne verrouille que le bytecode Python ; or $A @ B$ est exécutée par une bibliothèque native (BLAS, MKL, OpenBLAS) qui relâche le GIL et parallélise sur plusieurs cœurs. Une boucle Python pure, elle, garderait le GIL et ne profiterait pas du threading.

Q6. Le « Python sans GIL » existe-t-il, et faut-il compter dessus aujourd'hui ?

Réponse. Oui : une build free-threaded existe depuis Python 3.13 (officiellement supportée mais optionnelle en 3.14). Elle n'est pas la build par défaut, l'écosystème C n'est pas encore prêt, et le code mono-thread y est un peu plus lent. En 2026, le GIL reste donc présent par défaut.

Sur les conditions de course et les verrous

Q7. Qu'est-ce qu'une condition de course, et le GIL la prévient-il ?

Réponse. C'est quand plusieurs threads modifient une donnée partagée et que l'entrelacement de leurs opérations produit un résultat faux. Le GIL ne la prévient PAS : `solde += 1` est plusieurs instructions bytecode, et le basculement peut survenir au milieu, perdant des incréments.

Q8. Comment corrige-t-on une condition de course, et quel nouveau risque cela introduit-il ?

Réponse. Avec un verrou (`threading.Lock`), en entourant la section critique de `with verrou:` pour qu'un seul thread y entre à la fois. Le risque introduit est l'interblocage (deadlock) : deux threads attendant chacun le verrou de l'autre. Parade : acquérir les verrous toujours dans le même ordre.

Sur le multiprocessing et le matériel

Q9. Pour le multiprocessing, qu'est-ce qui compte : le nombre de CPU ou de cœurs ?

Réponse. Le nombre total de cœurs. Un ordinateur a presque toujours une seule puce (CPU) à plusieurs cœurs ; c'est le total de cœurs qui détermine le parallélisme réel.

Q10. 8 calculs de 10 s chacun : combien de temps sur 4 cœurs ? sur 16 cœurs ?

Réponse. Sur 4 cœurs : 2 vagues de 4 calculs = ~20 s. Sur 16 cœurs : les 8 tiennent en une vague = ~10 s.

Q11. Pourquoi `Pool(16)` sur seulement 8 calculs n'aide-t-il pas plus que `Pool(8)` ?

Réponse. Parce qu'il n'y a que 8 calculs à faire : 8 cœurs travaillent, les autres restent oisifs. On ne gagne rien à créer plus de processus que de tâches (ou que de cœurs).

Q12. Le multiprocessing aide-t-il sur une machine à 1 seul cœur ?

Réponse. Non. Les processus se relaient sur l'unique cœur, comme en série : aucun gain pour du calcul pur.

Sur `asyncio`, la comparaison et `concurrent.futures`

Q13. Comment fonctionne `asyncio` (la boucle d'événements) ?

Réponse. Une boucle, dans un seul thread, fait tourner les coroutines. Quand une coroutine atteint un `await` (attente), elle rend la main à la boucle, qui en lance une autre. Quand l'attente finit, la boucle y revient.

Q14. Quelle est l'incompatibilité d'`asyncio` que le `threading` n'a pas ?

Réponse. En `asyncio`, tout le code I/O doit être `async` de bout en bout : une seule opération bloquante (`time.sleep`, `requests` classique) gèle toutes les coroutines. Le `threading`, lui, accepte n'importe quel code existant.

Q15. `Threading` et `asyncio` : concurrence préemptive ou coopérative ?

Réponse. `Threading` = préemptif (le système d'exploitation bascule quand il veut), d'où le besoin de verrous. `asyncio` = coopératif (la main n'est cédée qu'aux `await`), ce qui réduit les conditions de course entre deux points d'attente.

Q16. À quoi sert `concurrent.futures`, et que renvoie `submit` ?

Réponse. À offrir une interface unique pour le `threading` (`ThreadPoolExecutor`) et le `multiprocessing` (`ProcessPoolExecutor`) : on change une classe, pas le code. `submit` renvoie un `Future`, objet « promesse » dont `.result()` donne la valeur (ou relève l'erreur) quand elle est prête.

Q17. `asyncio` ou `threading` accélèrent-ils un calcul pur ?

Réponse. Non, ni l'un ni l'autre : tous deux servent l'attente (I/O). Pour le calcul pur (CPU), il faut le `multiprocessing`.

Sur les queues

Q18. À quoi sert une queue entre tâches concurrentes, et pourquoi est-elle plus sûre que des verrous ?

Réponse. À échanger des données en sécurité (motif producteur-consommateur). Elle gère elle-même la synchronisation (verrous internes), sert de tampon, absorbe les pics et évite les pertes — sans qu'on ait à poser de `Lock` soi-même, donc moins de risque d'interblocage.

Q19. Pourquoi existe-t-il une queue différente par outil de concurrence ?

Réponse. Parce que chacun a ses contraintes : `queue.Queue` (`put/get` bloquants) pour les threads ; `asyncio.Queue` (avec `await`) pour les coroutines ; `multiprocessing.Queue` qui transfère les données entre processus (mémoire non partagée).

Q20. Pourquoi ne pas utiliser `queue.Queue` dans du code `asyncio` ?

Réponse. Parce que ses `put/get` sont bloquants et gèleraient le thread unique d'`asyncio`. Il faut `asyncio.Queue`, dont les opérations s'utilisent avec `await` pour rendre la main.

Document suivant (5/8) : la manipulation de données — NumPy (vectorisation), Pandas (DataFrames, séries temporelles) et la gestion des dates et fuseaux horaires.